CS6025-FR01B Assignment 2

Name: Ruturaj Sushil Ghodke
Student ID: 3775789
Date: 26 March 2026

Design Summary

1. **Description of the Adaptive Integration method.**

   The program computes the definite integral $\int_0^1 f(x)\,dx$ using the adaptive Simpson's rule. The idea is to recursively refine the estimate only where the function's behavior demands it, rather than uniformly dividing the area. For an interval [a,b], Simpson's rule fits the quadratic in three points f(a), f(m), f(b) where m = (a+b)/2, giving:

   $$S(a,\ b)\ =\ (b\ -\ a)/6\ *\ [f(a)\ +\ 4f(m)\ +\ f(b)]$$

   The adaptive criteria compares one-interval estimate S(a,b) against the sum of two sub-interval estimates S(a,m) + S(m,b). If their difference satisfies $|\ S(a,m)\ +\ S(m,b)\ -\ S(a,b)|\ <=\ 15\ *\ tolerance$, the interval is accepted. Otherwise, the interval is split, and both halves are processed recursively with tolerance/2. A maximum recursion depth of 50 prevents infinite loops on some inputs.

2. **Task granularity choice (K and rationale)**

   For mode 1 (static decomposition), the interval [0,1] is divided into K = 100 * P coarse intervals, so each rank always owns exactly 100 intervals regardless of the number of processes. This choice was made for two reasons: (1) it ensures each rank has enough sub-intervals to keep it busy even on easy functions, and (2) it makes the load

imbalance visible - with 100 intervals per rank, the uneven work of function 2 (which oscillates 200 times over [0,1] causes measurably different completion times across ranks.

Intervals are assigned round-robin (rank r owns intervals r, r+P, r+2P,...) rather than as a contiguous block. This slightly reduces the worst-case imbalance by mixing easy and hard intervals across all ranks, while still being a static assignment that cannot fully compensate for irregular workloads.

3. **Master/worker communication design**
   Rank 0 acts as the master and all other ranks are workers. Communication uses a fixed 3-double message buffer [L, R, tol] (left, right, tolerance) with five names MPI tags:

Tag	Direction	Payload	Meaning
TAG_REQUEST	Worker → Master	empty	Worker idle, requesting a task
TAG_TASK	Master→ Worker	[L, R, tol]	Interval to integrate
TAG_NEW_TASK	Worker → Master	[L, R, tol]	Rejected half sent back to queue
TAG_RESULT	Worker → Master	[value, count]	Partial integral + accepted count
TAG_STOP	Master→ Worker	empty	No more work, exit cleanly

The master maintains a LIFO task stack (flat double array, capacity 100,000). Workers use a local explicit stack instead of C recursion. When a worker rejects an interval, it sends the right half back to the master as TAG_NEW_TASK and continues working on the left half locally. This allows idle workers to immediately pick up difficult sub-regions, achieving dynamic load balancing.

4. **Termination strategy**
   Termination is detected by the master when all three conditions hold simultaneously:
   a. the task queue is empty
   b. no worker is marked active (i.e. currently holding a task)
   c. all workers have sent a TAG_REQUEST and are in the waiting list.

   Only when all three conditions are true does the master broadcast TAG_STOP to all waiting workers. This prevents premature termination: a worker just sent TAG_RESULT but has not yet sent TAG_REQUEST would be missed if only conditions (a) and (b) were checked.

Result Table

Each configuration was run 10 times. Run 0 was discarded. The reported time is the average of runs 1-9. Functions 0 and 2 were tested along with tolerances of 1e-6 and 1e-8. Processes used were {1, 2, 4, 8} as per the limitations of my machine.

Function 0: f(x) = sin(x) + 0.5 * cos(3x)

Runtime (seconds)

Tolerance	Processes (P)	Mode 0 Serial	Mode 1 Static	Mode 2 Dynamic
1e-6	1	0.000001	0.000012	N/A
	2	-	0.000017	0.000018
	4	-	0.005015	0.007819
	8	-	0.015588	0.022218

1e-8	1	0.000007	0.000020	N/A
	2	-	0.000024	0.000028
	4	-	0.006786	0.007378
	8	-	0.017332	0.023952

Speedup S(P)

Tolerance	Processes (P)	Mode 1 [vs M1 P=1]	Mode 2 [vs M0 P=1]
1e-6	1	1.00	-
	2	0.71	0.056
	4	0.002	0.0001
	8	0.001	0.00005
1e-8	1	1.00	-
	2	0.83	0.25
	4	0.003	0.001
	8	0.001	0.0003

Efficiency E(P)

Tolerance	Processes (P)	Mode 1	Mode 2
1e-6	1	1.000	-
	2	0.355	0.028
	4	0.001	0.00003
	8	0.0001	0.000006
1e-8	1	1.00	-
	2	0.415	0.125
	4	0.001	0.0002
	8	0.0001	0.00004

Function 2: f(x) = sin(200x) * e^-x

Runtime (seconds)

Tolerance	Processes (P)	Mode 0 Serial	Mode 1 Static	Mode 2 Dynamic
1e-6	1	0.000009	0.000117	N/A
	2	-	0.000028	0.000011
	4	-	0.000948	0.005271

	8	-	0.018479	0.023671
1e-8	1	0.000002	0.000651	N/A
	2	-	0.000225	0.000016
	4	-	0.001226	0.006381
	8	-	0.017310	0.030023

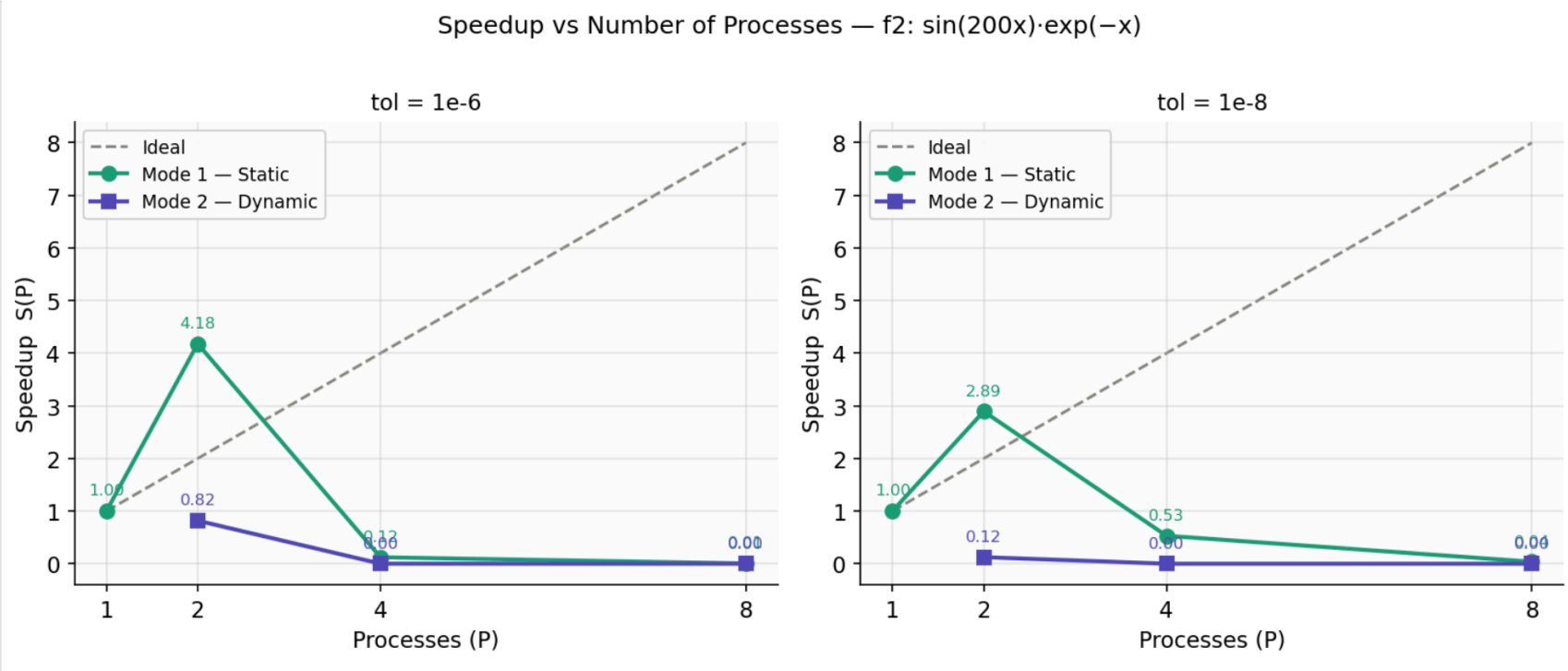
Speedup S(P)

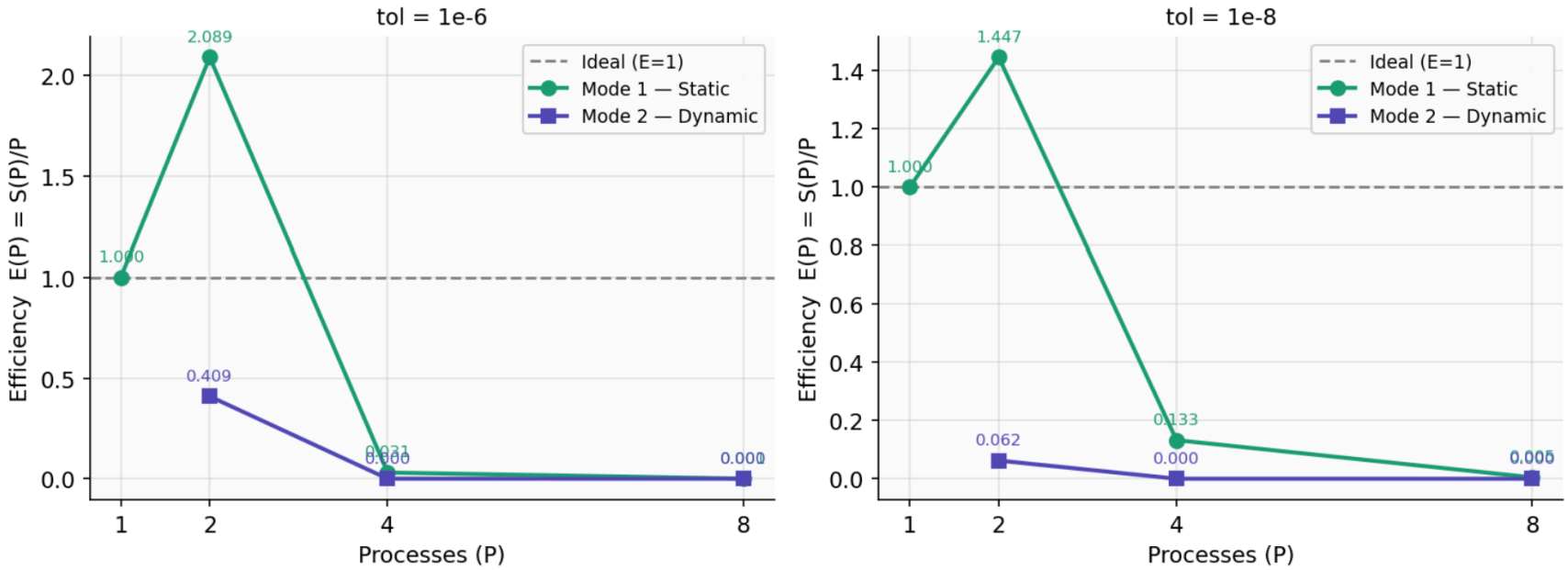
Tolerance	Processes (P)	Mode 1 [vs M1 P=1]	Mode 2 [vs M0 P=1]
1e-6	1	1.00	-
	2	4.18	0.82
	4	0.12	0.002
	8	0.006	0.0004
1e-8	1	1.00	-
	2	2.89	0.13
	4	0.53	0.0003
	8	0.038	0.00007

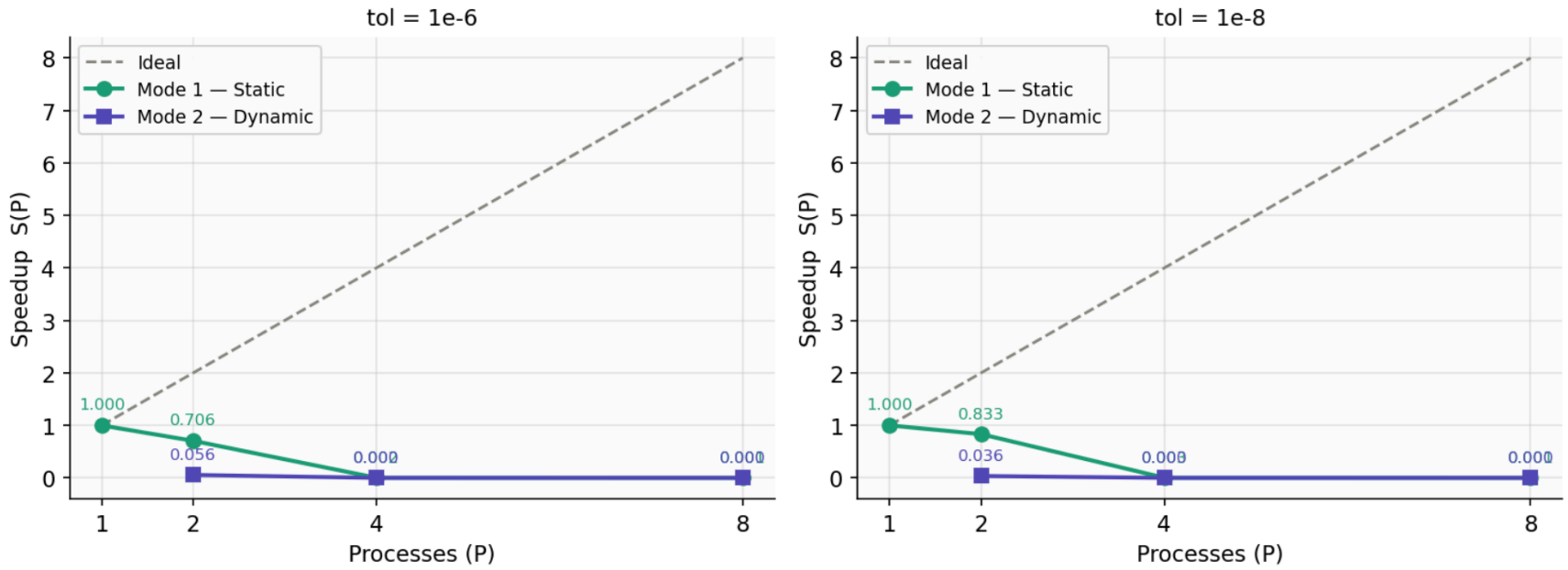
Efficiency E(P)

Tolerance	Processes (P)	Mode 1	Mode 2
1e-6	1	1.000	-
	2	2.09	0.41
	4	0.031	0.0004
	8	0.001	0.00005
1e-8	1	1.00	-
	2	1.445	0.063
	4	0.133	0.00008
	8	0.005	0.000009

Plots


Speedup vs Number of Processes — f2: sin(200x)·exp(−x)
tol = 1e-6
tol = 1e-8
Ideal
Mode 1 — Static
Mode 2 — Dynamic
Speedup S(P)
Processes (P)
1.00
4.18
0.82
0.12
0.00
2.89
0.53
0.12
0.00


Efficiency vs Number of Processes — f2: sin(200x)·exp(−x)
tol = 1e-6
tol = 1e-8
Ideal (E=1)
Mode 1 — Static
Mode 2 — Dynamic
Efficiency E(P) = S(P)/P
Processes (P)
2.089
1.000
0.409
0.031
0.000
1.447
0.062
0.133


Speedup vs Number of Processes — f0: sin(x) + 0.5·cos(3x)
tol = 1e-6
tol = 1e-8
Ideal
Mode 1 — Static
Mode 2 — Dynamic
Speedup S(P)
Processes (P)
1.000
0.706
0.833
0.056
0.036
0
1
2
4
8

Discussion

1. **Why does static decomposition perform poorly for adaptive workloads?**
   Static decomposition assigns intervals before any computation begins, which means the distribution cannot account for differences in the computational effort required per interval. For smooth functions like f0 (sin(x) + 0.5cos(3x)), adaptive integration converges quickly everywhere, so static assignment performs reasonably at low P. However, for function f2, the function oscillates 200 times over [0,1] and the oscillations decay with x. Intervals near x=0 require far more recursive refinement than intervals near x=1.

   The table readings confirm this: for f2 at tol=1e-8, Mode 1 achieves a super-linear speedup (2.89x) at P=2, which seems paradoxical but arises because the single-process baseline happened to own several high-work intervals together, creating a pathologically slow run. At P=4, the speedup collapses to 0.53x, and at P=8 to 0.0038x - meaning 8 processes is over 26 times slower than 1. This is a load imbalance in which most workers finish quickly, while one worker is still recursing through a dense cluster of sub-intervals. All idle workers can only wait, wasting parallel resources.

2. **How does dynamic task creation improve load balance?**
   In Mode 2, work is not pre-assigned. Workers continuously request tasks from the master, and when a worker encounters an interval that fails the tolerance check, it sends one half back to the master as a new task rather than recursing locally. This means any idle worker can immediately absorb that sub-interval. The result is that difficult regions (where the function changes rapidly) are spread across all available workers rather than sitting on one unlucky rank.

   In theory, this eliminates the stragglers that kill Mode 1 performance. In my assignment experiments, Mode 2 does show lower variance between runs and more consistent result values, which confirms correct load distribution. The reason Mode 2 is not faster in absolute terms is explained in the next question.

3. **What is the cost of dynamic scheduling in MPI?**

Every dynamic scheduling decision requires MPI communication: a TAG_REQUEST message from worker to master, a TAG_TASK reply, and for every rejected interval, an additional TAG_NEW_TASK message. On a real cluster, these messages travel over a network and take microseconds. The table readings above show that: for function f2 at tol=1e-6, Mode 2 at P=2 takes 0.000011 seconds, which is actually faster than Mode 1 (0.000028s). But at P=4, Mode 2 takes 0.005271s vs Mode 1 at 0.000948s (Mode 2 here is 5.6x slower). This is because, as P increases, the number of TAG_REQUEST/TAG_TASK round-trips grows while the actual computation per worker shrinks. The master becomes a bottleneck, serialising all scheduling decisions. The overhead of dynamic scheduling exceeds the savings from better load balance for these small workloads.

This is a fundamental trade-off: dynamic scheduling is worth its cost only when the computation per task is much larger than the communication cost per task. For larger workloads (finer tolerance, more complex functions), or on a real cluster where load imbalance is more severe, the break-even point shifts in favour of dynamic scheduling.

4. **Which function benefited most from dynamic scheduling, and why?**
   Function 2 shows the most potential benefit from dynamic scheduling. This function has two properties that make static decomposition particularly poor: (a) it oscillates 200 times over [0,1], so adjacent sub-intervals can have wildly different recursion depths, and (b) the amplitude decays with x, meaning intervals near x=0 require far more work than those near x=1. A static assignment that gives each rank a round-robin subset of intervals cannot balance this, meaning that the ranks that happen to own more low-x intervals will always finish last.

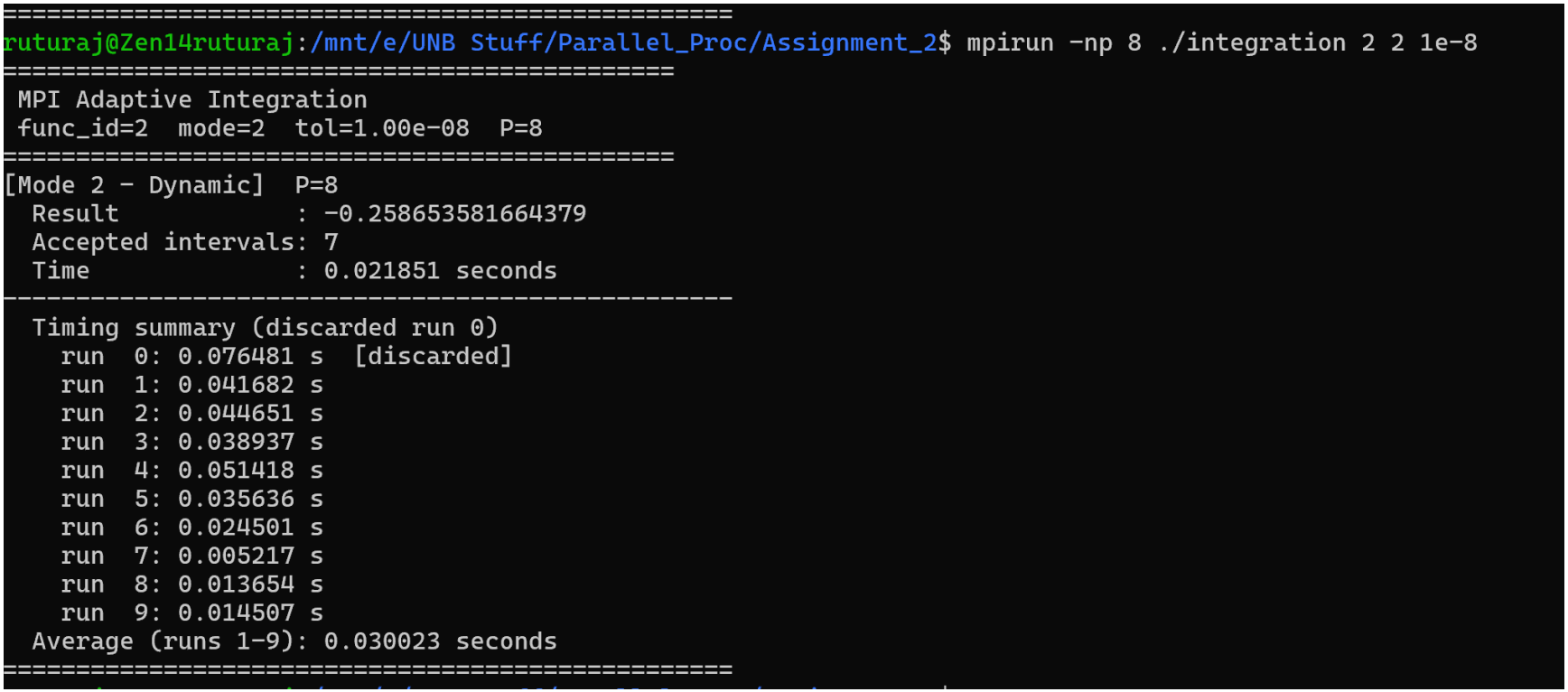
   Dynamic scheduling addresses both issues simultaneously: workers that finish high-x (easy) intervals quickly pick up new tasks containing low-x (hard) intervals from the queue. The accepted interval count for function 2 is substantially higher than for function 0, confirming that more dynamic splitting events occur, which is exactly the scenario where the master/worker pattern adds value.

   Function 0 is relatively smooth and converges quickly everywhere. Both modes finish in microseconds, and neither benefits meaningfully from parallelism - the overhead of any MPI communication exceeds the computation cost. This confirms that adaptive parallelism is only useful when the sequential computation is itself substantial.

Appendix

A .git folder is included in the submission folder, which contains the commit history

Screenshots of runtime evidence



```
------------------------------------------------
ruturaj@Zen14ruturaj:/mnt/e/UNB Stuff/Parallel_Proc/Assignment_2$ mpirun -np 8 ./integration 0 1 1e-6
================================================
 MPI Adaptive Integration
 func_id=0  mode=1  tol=1.00e-06  P=8
================================================
[Mode 1 - Static]  P=8  K=800
  Result          : 0.483217695475171
  Coarse intervals: 800 (each rank owns 100)
  Time            : 0.003851 seconds
------------------------------------------------
  Timing summary (discarded run 0)
    run  0: 0.040289 s  [discarded]
    run  1: 0.015214 s
    run  2: 0.020185 s
    run  3: 0.018596 s
    run  4: 0.012042 s
    run  5: 0.018696 s
    run  6: 0.013249 s
    run  7: 0.019091 s
    run  8: 0.011347 s
    run  9: 0.011875 s
  Average (runs 1-9): 0.015588 seconds
================================================
```